Bergen Traffic Predictor

TODO: Navn på gruppemedlemmer

24. oktober 2026

Kort sammendrag. Prosjektets opprinnelige spørsmål var om historiske weather features fra Frost kunne forbedre prediction av biltrafikk én time frem ved Danmarks plass. Kronologiske eksperimenter med både direkte lag features og en avledet weather feature-pakke ga ingen forbedring; traffic-only models presterte bedre. Sluttproduktet er derfor en lokal Streamlit-demonstrasjon med en frozen model uten weather features, basert på kalender og historisk trafikk. Modellen oppnådde MAE 108,81 kjøretøy/time på den tidligere ubrukte perioden 2026 H1, mot 211,79 for forrige-uke-baselinen på de samme 4 343 tidspunktene. Den negative værkonklusjonen er et sentralt resultat. Løsningen er et proof of concept for ett målepunkt, ikke en offentlig driftet tjeneste.

Innhold

1 Beskriv problemet **2**

1.1 Omfang . . . 2

1.2 Metrikker . . . 3

2 Data **3**

2.1 Kilde, periode og labels . . . 3

2.2 Rensing og målegap . . . 4

2.3 Feature engineering . . . 4

2.4 Personvern og etikk . . . 5

3 Modellering **5**

3.1 Fra værhypotese til frozen candidate . . . 5

3.2 Kronologisk evaluering . . . 6

3.3 Feilanalyse . . . 6

4 Deployment **7**

4.1 Bygging, lagring og lasting . . . 7

4.2 Brukerflyt og prognose . . . 7

4.3 Feilhåndtering, demonstrasjon og drift . . . 7

5 Referanser **8**

1 Beskriv problemet

1.1 Omfang

Bergen Traffic Predictor estimerer hvor mange kjøretøy som passerer ett bestemt trafikkregistreringspunkt i løpet av neste time. Punktet er Danmarks plass med ID `14630V805616`. Hver modellrad representerer et tidspunkt t, mens target er observert trafikkvolum ved $t + 1$. Oppgaven er dermed supervised regression og kortsiktig time-series forecasting.

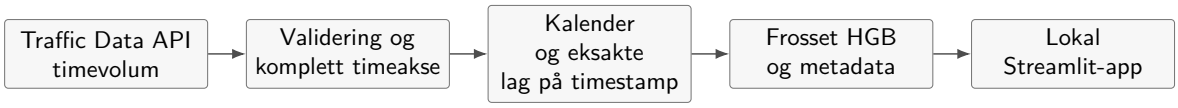
Den opprinnelige prosjektideen var ikke en ren traffic-only model. Hypotesen var at vær påvirker reisemønsteret, og at historiske weather features fra MET Norways Frost API derfor kunne forbedre prediction sammenlignet med trafikkhistorikk alene. Prosjektets ML lifecycle undersøkte denne hypotesen med både direkte weather lag features og avledede weather features. Ingen av variantene ga bedre kronologisk validation enn modellen uten vær. Den endelige modellen uten weather features er derfor et resultat av eksperimentene og modellvalget, ikke en avgrensning som ble bestemt på forhånd.

Målgruppen i et videreutviklet system kan være trafikkplanleggere, analytikere og andre som trenger et enkelt anslag for belastningen ved punktet. En prognose kan for eksempel støtte bemanning, midlertidig trafikkstyring eller vurdering av avvik fra et normalt ukemønster. Studentprosjektet dokumenterer imidlertid ingen faktisk beslutningsbruk, brukertall eller økonomisk gevinst. Vi vurderer det derfor som et proof of concept: systemet viser en komplett kjede fra åpne data til en interaktiv prognose, men gir ikke grunnlag for å hevde produksjonsverdi eller lønnsomhet.

En manuell prognose kan lages ved å slå opp trafikken på samme klokkeslett én uke tidligere. Dette er både forståelig og operasjonelt enkelt, og er prosjektets sentrale baseline. En analytiker kunne i tillegg justert tallet skjønnsmessig for helg, helligdag eller ferie. Machine learning er relevant fordi trafikkmønsteret varierer systematisk med time, ukedag og kalenderhendelser, og fordi sammenhengen mellom flere lag features kan være non-linear. Modellen kan gjøre den samme beregningen konsekvent for alle timer, men den erstatter ikke lokal kunnskap om ulykker, vegarbeid eller arrangementer som ikke finnes i datasettet.

Avgrensningen til ett punkt og én times horisont var bevisst. Den gjør det mulig å kontrollere hele dataløpet og sammenligne modellene på en entydig oppgave innenfor prosjektperioden. Modellen sier ikke noe om reisetid, kølengde, retning eller årsaken til et trafikkvolum. Den kan heller ikke overføres direkte til et annet målepunkt, fordi døgnprofil, vegtype og lokale hendelser kan være annerledes. En utvidelse må derfor behandles som en ny modelloppgave, ikke som en enkel endring i appens punkt-ID.

Figur 1 viser systemgrensen. Statens vegvesens Traffic Data API leverer aggregerte trafikkmålinger; API-et har én time som korteste offentlige intervall og publiserer kvalitetsparametere sammen med dataene [1]. Prosjektet validerer og normaliserer serien, bygger features på eksakte tidspunkter, trener en frozen model og bruker den i en lokal Streamlit-app. Endringer i datakvalitet eller publiseringsforsinkelse påvirker både feature availability og om appen kan lage en prognose. Featurekode deles derfor mellom trening og app, slik at samme definisjon brukes i begge ledd.



Figur 1: Implementert pipeline fra offentlig trafikkdata til lokal demonstrasjon.

Prosjektets suksesskriterium er teknisk og avgrenset: en leakage-free model skal slå en relevant forrige-uke-baseline på en senere, urørt periode, og hele løsningen skal kunne bygges fra dokumenterte kommandoer. Appen skal gi en forståelig prognose eller en tydelig feilmelding. Markedsmessige

suksessmål er ikke definert fordi de ikke er undersøkt.

Ressursbehovet er moderat. Flerårige timedata for ett punkt får plass lokalt, og HGB kan trenes på en vanlig bærbar datamaskin. Den viktigste ressursen har vært utviklingsarbeid: datavalidering, lekkasjekontroll, kronologisk evaluering og integrasjon mellom modell og app. En produksjonsløsning ville i tillegg krevd driftsansvar, overvåking og rutiner for modellgodkjenning.

1.2 Metrikker

Hovedmetrikken er gjennomsnittlig absolutt feil, MAE,

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{1}$$

MAE måles i kjøretøy per time og kan derfor tolkes direkte. Lavere verdi er bedre, og null er perfekt samsvar. RMSE rapporteres som støttemål fordi kvadreringen gir større utslag for enkelte store feil:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{2}$$

Alle modell- og baselinemål beregnes på identiske forecast-rader. Dette er viktig fordi manglende lag ellers kunne gitt én metode et enklere utvalg. Vi rapporterer også antall evaluerte prediksjoner, feil etter forecast-time og feil etter ukedag. For programvaren kontrolleres reproduserbar bygging, gyldig modellartefakt, tilgjengelige lag, API-feil og tidssonehåndtering. Prosjektet har ikke målt svartid, oppetid eller brukertilfredshet; slike mål hører til en eventuell senere driftstest.

2 Data

2.1 Kilde, periode og labels

Trafikkdata hentes fra Statens vegvesens åpne Traffic Data API, som inneholder aggregerte registreringer fra offentlige veger [2]. Uttrekket dekker 1. januar 2022 til 1. juli 2026 i lokal tid og gjelder feltet «Totalt» ved punkt `14630V805616`. Den normaliserte timeaksen har 39 407 timer. Av disse har 38 956 observert trafikkvolum og 451 mangler verdi. Perioden 2022–2025 har 35 064 timer, 34 613 observasjoner og alle de 451 manglene. Den reserverte perioden 2026 H1 har 4 343 observerte timer og ingen mangler.

Det opprinnelige API-uttrekket inneholder 39 025 rader. Kvalitetsrapporten markerer 69 av dem som ugyldige fordi volum mangler, men finner ingen dupliserte timestamps. Når datasettet reindekseres til alle forventede UTC-timer, blir både disse radene og timer uten API-rad stående som ukjente verdier. Forskjellen mellom rått radantall og 39 407 timer på den komplette aksen er derfor forventet og dokumentert, ikke et tap som oppstår under behandling.

API-resultatet lagres først som rå CSV sammen med kilde, hentetidspunkt, punkt-ID, periode og radantall. Det prosesserte datasettet får en sidecar-fil med SHA-256 for råfilen, kvalitetsrapport og kolonneliste. Rådata, prosesserte data og modellartefakter er utelatt fra Git, men kan gjenskapes med `make build-app-model`. Denne kommandoen henter hele perioden, bygger datasettet og trener sluttmodellen.

Labelen oppstår uten manuell annotering: for en feature-rad ved t slås `traffic_volume` ved nøyaktig $t+1$ opp som `target_traffic`. Trafikkmåleren er dermed både datakilde og fasit. Dette

gir konsistente enheter, men ikke nødvendigvis en feilfri sannhet; sensorfeil og ufullstendig dekning kan fortsatt påvirke både features og labels. API-ets kvalitetsinformasjon lagres i rådataene, mens denne prosjektversjonen primært bruker tilstedeværelse, gyldig timestamp og ikke-negativt volum som valideringskrav.

2.2 Rensing og målegap

Innlest tid konverteres til tidssonebevart UTC, sorteres og kontrolleres for ugyldige og dupliserte tidspunkter. Kalenderinformasjon beregnes etter konvertering til `Europe/Oslo`. Denne arbeidsdelingen bevarer entydige tidspunkt gjennom sommertid og vintertid samtidig som time og dato får riktig lokal betydning.

Det finnes to dokumenterte målegap: 67 timer fra 4. juni 2022 kl. 21 til 7. juni kl. 15, og 384 timer fra 2. oktober 2024 kl. 10 til 18. oktober kl. 09, alle tider i `Europe/Oslo`. Pipelinen oppretter en komplett timeakse og lar disse 451 verdiene stå som manglende. De blir ikke satt til null, interpolert eller erstattet. Null ville betydd observert fravær av trafikk, mens en manglende verdi betyr at volumet er ukjent. Minimum og maksimum blant observerte timer i hele datasettet er henholdsvis 40 og 4 235 kjøretøy; ingen observerte timer er null.

Den innledende EDA-en viste et tydelig døgn- og ukemønster. For 2022–2025 var gjennomsnittet høyest kl. 15 med 3 426 kjøretøy per time og lavest kl. 03 med 185. Helgesnittet var 1 275, som var 32,5 prosent lavere enn hverdagssnittet på 1 889. Disse mønstrene begrunner kalenderfeatures og den ukentlige baselinen, men de brukes ikke som en alternativ fasit. Volumfordelingen alene sier heller ikke om en høy verdi er normal for en bestemt dato.

Alle lag og targets hentes ved eksakt timestamp, aldri ved radforskyvning. Hvis en feature krever $t - 24$ og denne timen ligger i et målegap, blir featuren manglende selv om forrige tilgjengelige rad finnes 25 timer tilbake. Modellrader med manglende nødvendig trafikklag eller target fjernes før trening og evaluering. Dette er et dokumentert utvalg, ikke imputering, og hindrer at observasjoner flyttes over hull i serien.

2.3 Feature engineering

Sluttmodellen bruker 14 features. Seks beskriver lokal kalender: time, ukedag, måned, helg samt sinus og cosinus av time. Fem deterministiske indikatorer markerer offentlig helligdag, dag før og etter helligdag, jul- og nyttårsperioden og 17. mai. De tre numeriske traffic lag features er observert volum ved nøyaktig $t - 24$, $t - 48$ og $t - 168$. Alle er kjent eller kan konstateres som manglende ved prognosetidspunktet. Den tree-based sluttmodellen krever ingen scaling.

Sinus og cosinus gjør at kl. 23 og kl. 00 ligger nær hverandre i feature-representasjonen. Helligdagsindikatorene beregnes fra kalenderen og krever ingen observerte fremtidsdata. Trafikklagene følger derimot absolutte tidsforskjeller i UTC. Dermed endres ikke den faktiske avstanden ved overgang mellom sommer- og vintertid, selv om det lokale klokkeslettet kan være annerledes.

Værdataene ble hentet fordi prosjektets opprinnelige hypotese var at weather features kunne forbedre prediction utover det historisk trafikk alene ga. Historiske observasjoner kom fra MET Norways Frost API for Bergen–Florida (SN50540). Frost gir tilgang til arkiverte observasjoner og krever kilde, referansetid og element i forespørselen [3]. Temperatur og nedbør dekket 35 016 av 35 064 timer i 2022–2025, mens vind dekket 34 458. Prosjektet brukte konservative weather lag features fordi `referenceTime` beskriver observasjonstid, ikke publiseringstid [4]. Både de direkte weather lag features og en senere avledet weather feature-pakke ga svakere validation enn traffic-only model. Det sentrale forsøket på å forbedre modellen med vær lyktes dermed ikke, og Frost inngår derfor ikke i sluttmodellen eller appen.

2.4 Personvern og etikk

Datasettet inneholder aggregerte passeringstall per time, ikke kjøretøy- eller personidentifikatorer. Personvernrisikoen er derfor lav. Den viktigste etiske risikoen er feilaktig bruk: ett registreringspunkt representerer ikke all trafikk på Danmarks plass eller Bergen, og en prognose kan være misvisende ved hendelser som modellen ikke kjenner. Appen oppgir derfor punkt-ID, datakilde og begrensning. En senere operativ løsning bør i tillegg dokumentere datalisens, tilgjengelighet, sensorendringer og hvem som har ansvar for beslutninger som støttes av prognosen.

3 Modellering

3.1 Fra værhypotese til frozen candidate

Utviklingen fulgte en kronologisk ML lifecycle med værhypotesen som utgangspunkt. Målet var å undersøke om weather features kunne gi lavere feil enn modeller basert på historisk trafikk. Først ble forrige-uke-prediksjonen etablert som kontroll. For en forecast med target $t+1$ bruker baselinen observasjonen ved $(t+1)-168 = t-167$, altså samme target-time én uke tidligere. En lineær regresjon og en `HistGradientBoostingRegressor` (HGB) ble deretter trent med kalender og traffic lag features ved $t-1$, $t-2$, $t-24$ og $t-168$. HGB er en histogram-based gradient boosting model for tabulære data. På validation i 2024 fikk lineær regresjon MAE 322,45, HGB 103,28 og baselinen 200,78. HGB ble derfor valgt og oppnådde MAE 106,22 på 2025-testen.

Sammenligningen brukte 8 231 felles rader i 2024 og 8 760 i 2025. Modellen ble først tilpasset på 2022–2023 for validation. Etter valg ble HGB trent på training- og validation-data samlet før den ene målingen på 2025-testen. Denne rekkefølgen skiller mellom model selection og å anslå generalization error. Dersom 2025 hadde blitt brukt til å endre features etter hver måling, ville perioden gradvis blitt en ny validation period og sluttresultatet for optimistisk.

Videre analyse viste at store feil ofte sammenfalt med helligdager og høytider. Deterministiske norske kalenderindikatorer ble derfor vurdert med tre expanding folds i 2022–2023. Gjennomsnittlig CV-MAE ble 119,69 med indikatorene og 120,03 uten. Deretter ble den opprinnelige værhypotesen testet direkte. En raw weather model tapte mot traffic-only model på felles 2024-rader, med MAE 106,73 mot 103,41. En forhåndsdefinert weather feature-pakke med blant annet akkumulert nedbør og sesongavvik for temperatur og vind tapte senere alle tre CV-foldene: 124,43 mot 119,69. Vær ga altså ingen dokumentert forbedring i noen av de to forsøkene og ble forkastet før sluttmodellen ble bestemt.

Den første frozen calendar HGB-en oppnådde MAE 102,35 på 2025, men krevde de ferske lag features $t-1$ og $t-2$. Traffic Data API kan publisere timeaggregater med forsinkelse, så modellen var lite egnet til en reell neste-time-forespørsel. Prosjektet definerte derfor tre små, latency-robust lagpakker på forhånd: $(24, 168)$, $(24, 48, 168)$ og $(24, 168, 336)$. Alle brukte de samme calendar feature-typene og HGB med fast `random_state=42`. Gjennomsnittlig CV-MAE i 2022–2023 ble henholdsvis 134,88, 133,77 og 135,80. Pakken $(24, 48, 168)$ ble fryst som `traffic-hgb-latency-robust-v1`.

Den latency-robust modellen er ikke kandidaten med lavest tidligere 2025-MAE; den løser et annet programvarekrav. Ved å fjerne $t-1$ og $t-2$ kan appen lage en prognose selv om de nyeste aggregatene ikke er publisert. Kostnaden er at modellen mister informasjon om den umiddelbare trafikkutviklingen. Kandidatlisten ble holdt liten for å begrense tilpasning til CV-foldene, og ingen hyperparameters ble valgt fra 2026-resultatet.

3.2 Kronologisk evaluering

Random splitting ville blandet fortid og fremtid og gitt en misvisende vurdering av tidsserien. Prosjektet brukte derfor time-series cross-validation der hver validation period ligger etter training period. Model selection brukte expanding training windows med validation i 2022 H2, 2023 H1 og 2023 H2. Perioden 2025 var allerede brukt som test set for en tidligere modell og ble derfor låst for videre valg. Etter at den latency-robust kandidaten var frozen, ble den trent på 2022–2025 og evaluert én gang på det nye test set-et 2026 H1. Sluttresultatet ble ikke brukt til ny tuning.

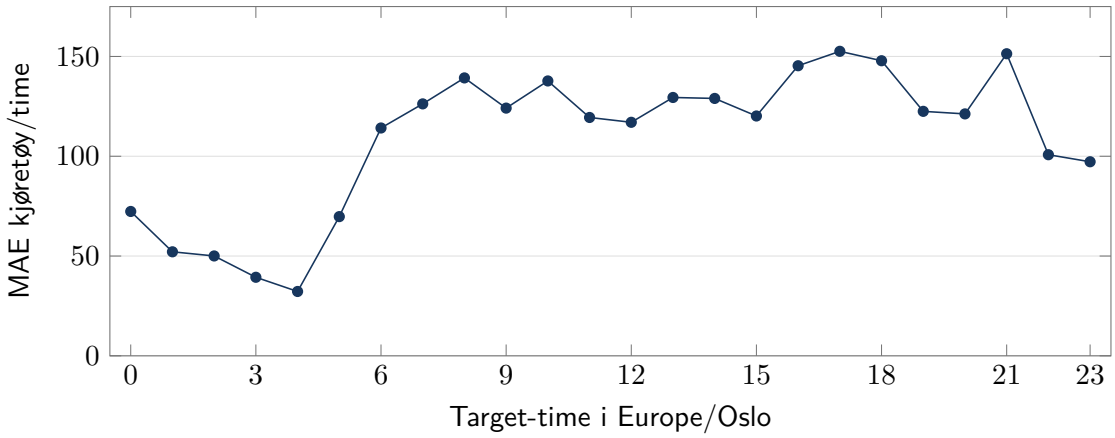
Tabell 1 sammenligner modell og baseline på nøyaktig de samme 4 343 radene. Modellen reduserer MAE med 102,98 kjøretøy per time, tilsvarende 48,6 prosent relativt til baselinen. RMSE er også vesentlig lavere, men ligger over MAE fordi enkelte timer har store feil.

Tabell 1: Sluttevaluering på den tidligere ubrukte perioden 2026 H1.

Modell	MAE	RMSE	Rader
Latensrobust HGB	108,81	161,85	4 343
Forrige uke	211,79	454,47	4 343

3.3 Feilanalyse

Figur 2 viser MAE etter target-time i 2026 H1. Feilen er lavest kl. 04 med 32,24 og høyest kl. 17 med 152,58 kjøretøy per time. Perioden fra kl. 07 til 18 er gjennomgående vanskeligere enn natten, noe som er rimelig når volumet og endringstakten er større. Kl. 21 har også høy MAE, 151,36, og viser at utfordringen ikke bare kan reduseres til klassisk rushtrafikk.



Figur 2: Sluttmodellens MAE per target-time i 2026 H1. Figuren genereres fra prosjektets dokumenterte sluttrapport.

Etter ukedag varierer MAE fra 97,05 på tirsdag til 121,89 på lørdag. De øvrige dagene ligger mellom 100,14 og 112,98. Analysen viser når feilen er størst, men forklarer ikke årsaken alene. Prosjektet har ikke beregnet feature importance og rapporterer derfor ingen konstruert rangering. Aktuelle forbedringer må testes i en ny kronologisk valideringsrunde, for eksempel hendelsesdata eller bedre mål på publiseringsforsinkelse, før en ny og senere testperiode åpnes.

RMSE 161,85 mot MAE 108,81 viser at fordelingen inneholder noen større avvik. For en bruker betyr dette at gjennomsnittsfeilen ikke bør leses som en garanti for en enkelt time. Appen viser derfor et tallfestet anslag og historikk, men ingen sikkerhetskategori som kunne gitt et falskt inntrykk av kalibrert usikkerhet.

4 Deployment

4.1 Bygging, lagring og lasting

Sluttproduktet er en lokal Streamlit-demonstrasjon, ikke en offentlig driftet tjeneste. Repositoryets `make build-app-model` kjører tre eksplisitte steg: nedlasting av trafikkdata, normalisering med bevarte målegap og training av den frozen kandidaten. Training-scriptet gjenskaper CV-rapporten og sluttevalueringen, trener modellen på alle gyldige rader før 1. januar 2026 og lagrer et Joblib-bundle i `models/traffic_hgb_latency_robust_v1.joblib`. Bundle-filen inneholder den trente HGB-en, den ordnede feature-listen og metadata om modellversjon, lagpakke, utvalgsperiode, treningsperiode og datakilde. Den tilhørende JSON-filen viser 34 159 treningsrader.

Joblib gir `dump` og `load` for serialisering av Python-objekter [5]. Artefakten er ignorert av Git og må bygges lokalt. Appen laster bare den forventede lokale filen og gir en konkret byggekommando dersom den mangler eller ikke kan leses. Fordi Joblib bygger på pickle, bør en operativ løsning aldri laste en modellfil fra en uklar eller ukontrollert kilde.

4.2 Brukerflyt og prognose

Appen startes med `streamlit run app/streamlit_app.py`. Streamlit er et Python-rammeverk for interaktive dataapplikasjoner [6]. Brukeren velger enten «Neste time» eller et historisk tidspunkt og trykker «Vis prognose». Historiske valg begrenses til reelle lokale klokketimer: begge forekomster av en gjentatt time ved vintertid kan velges, mens en ikke-eksisterende time ved sommertid ikke tilbys. Prognoseintervallet vises med UTC-offset og `Europe/Oslo`.

Ved hvert klikk lastes modellen, og appen henter nødvendig historikk fra Traffic Data API. Target-tidspunktet kalles $t + 1$, og feature-ankeret er t. Featurekoden bygger kalenderverdier og slår opp trafikk ved $t - 24$, $t - 48$ og $t - 168$. Hvis en av disse verdiene mangler, blir det ingen prediksjon. Hvis alle finnes, viser appen forventet biltrafikk i kjøretøy per time, en graf med de siste 72 timene og en separat diamant for prognosen. Målegap og upubliserte timer vises som åpne hull, uten linjer som skjuler manglende data.

Appens sammenligning med forrige uke må ikke forveksles med modellfeaturen. UI-et viser observert trafikk ved *target minus 168 timer*. Siden target er $t + 1$, tilsvarer dette $t - 167$. Dette er samme tidsposisjon som den operative forrige-uke-baselinen bruker. Modellens feature ved $t-168$ er derimot knyttet til feature-ankeret og ligger 169 timer før target. Verdiene har derfor ulike timestamps, selv om begge omtales som ukentlig historikk. Ved sommertid og vintertid kan også lokalt klokkeslett forskyves når forskjellen beregnes som eksakt forløpt tid.

Sammenligningsverdien er valgfri: mangler den, kan en gyldig modellprognose fortsatt vises. Prosentforskjellen beregnes bare når sammenligningen er positiv. En utfellbar seksjon viser modellnavn, features, evalueringsresultat og de konkrete inputverdiene. Datastatus viser kilde, hentetidspunkt og siste tilgjengelige observasjon i uttrekket.

4.3 Feilhåndtering, demonstrasjon og drift

Appen skiller mellom manglende eller skadet modellartefakt, feil fra API-et og utilstrekkelig trafikkhistorikk. Brukeren får et kort neste steg, mens tekniske detaljer om manglende input kan åpnes ved behov. Hvert klikk gjør et nytt API-kall; det brukes ingen cache eller lokal database i denne versjonen. Historiske prognoser bruker data som er tilgjengelige nå. De rekonstruerer ikke hva API-et hadde publisert på det historiske prognosetidspunktet, og prognoser innen treningsperioden er demonstrasjoner, ikke nye testresultater.

En videodemonstrasjon er lenket fra prosjektets README og oppfyller oppgavens krav om å vise nettsiden uten offentlig hosting: Bergen Traffic Predictor videodemonstrasjon. Løsningen kan kjøres på en vanlig lokal datamaskin med Python 3.11 eller nyere. Den krever ingen Frost-nøkkel fordi vær ikke inngår i sluttproduktet.

Før reell drift må systemet få vedvarende logging og monitorering. Minstekrav er API-feilrate, andel prognoser som mangler ett eller flere lag, alder på siste observasjon og endringer i fordelingen av trafikkvolum. Når target senere blir observert, bør MAE og RMSE følges over tid og fordeles på time og ukedag. Vedvarende avvik kan skyldes datadrift, sensorendring, vegomlegging eller endret reisemønster. Modellen bør trenes på nytt etter en fast vurderingsplan eller ved dokumentert drift, men en ny versjon må velges på kronologisk validering og måles på en ny, urørt periode. API-tilgjengelighet, datalisens, artefaktintegritet og en manuell reserveprognose må også inngå i en produksjonsplan.

Monitoreringen bør skille datafeil fra modelldrift. Manglende lag eller en plutselig økning i API-feil peker mot datakjeden, mens stabile input med økende prediction error kan tyde på model drift. Modell- og dataversjon bør logges sammen med hver prognose slik at en feil kan spores. Før automatisk retrening innføres, må det finnes en godkjenningsmekanisme som hindrer at et midlertidig målegap eller en unormal periode blir gjort til ny normal.

5 Referanser

Eksterne kilder nedenfor dokumenterer datatjenestene og bibliotekene som løsningen bygger på. Alle numeriske prosjektresultater er egne, reproduserbare eksperimenter fra disse repositoryfilene:

- `docs/traffic-eda-baseline.md`
- `docs/initial-ml-model.md`
- `docs/final-model-evaluation.md`
- `docs/latency-robust-model-evaluation.md`

Filene genereres av skriptene i `scripts/` og skal ikke tolkes som resultater fra de eksterne referansene.

[1] Statens vegvesen. *Trafikkdata-API*. Beskrivelse av innhold, tidsoppløsning, kvalitetsparametere og lisens. URL: https://dataut.vegvesen.no/nb/dataset/trafikkdata/resource/968a10a7-f704-4cd7-9795-de69e8cad2bc (sjekket 24.10.2026).

[2] Statens vegvesen. *Trafikkdataportalen*. URL: https://trafikkdata.atlas.vegvesen.no/ (sjekket 24.10.2026).

[3] MET Norway. *Frost API How to use Frost*. URL: https://frost.met.no/howto.html (sjekket 24.10.2026).

[4] MET Norway. *Frost API Relationship between referenceTime timeOffset and timeResolution*. URL: https://frost.met.no/concepts2.html (sjekket 24.10.2026).

[5] Joblib developers. *Serialization and memory-mapped loading*. URL: https://joblib.readthedocs.io/en/latest/user_guide/persistence.html (sjekket 24.10.2026).

[6] Streamlit. *Streamlit documentation*. URL: https://docs.streamlit.io/ (sjekket 24.10.2026).